

ASSIGNMENT-17.1

Name: B.Sherlin Varshitha

H.T.NO: 2303A52266

Batch: 36

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd  
data = pd.read_csv("shopping_trends.csv")  
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

Commands + Code + Text Run all

Files

- sample_data
- archive (7).zip
- dirty_v3_path.csv
- sales_data.csv
- shopping_trend...

```

# Scale numerical features using MinMaxScaler
scaler = MinMaxScaler()

# Identify numerical columns to scale (excluding original categorical and 'id')
# Also exclude newly created encoded columns from scaling
all_cols = set(healthcare_df.columns)
encoded_cols = {col for col in healthcare_df.columns if '_Encoded' in col}
original_categorical_cols = set(categorical_cols)

# Numerical columns are those that are not object type, not encoded, and not 'id'
numerical_cols = [col for col in healthcare_df.select_dtypes(include=np.number).columns
                  if col not in encoded_cols and col != 'id']

for col in numerical_cols:
    healthcare_df[col + '_Scaled'] = scaler.fit_transform(healthcare_df[[col]])

print("\nDataFrame Head after scaling numerical features:")
display(healthcare_df.head())

```

DataFrame Head after scaling numerical features:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	Lengthofstay	Cholesterol	Triglycerides	...	Triglycerides_Scaled	HbA1c_Scale
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	...	0.524877	0.47687
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	...	0.342102	0.17951
2	50.0	NaN	Asthma	95.24	138.32	22.53	90.31	2	214.94	165.35	...	0.423050	0.25550
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	...	0.460844	0.40088

Variables Terminal

1:47 PM Python 3

Commands + Code + Text Run all

```

8 Color 3900 non-null object
9 Season 3900 non-null object
10 Review Rating 3900 non-null float64
11 Subscription Status 3900 non-null object
12 Payment Method 3900 non-null object
13 Shipping Type 3900 non-null object
14 Discount Applied 3900 non-null object
15 Promo Code Used 3900 non-null object
16 Previous Purchases 3900 non-null int64
17 Preferred Payment Method 3900 non-null object
18 Frequency of Purchases 3900 non-null object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB

```

Original DataFrame Head:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal

Variables Terminal

1:47 PM Python 3

Now, let's proceed with cleaning the dataset. We'll start by handling missing values in the specified columns.

```

# Handle missing values
# For numerical columns ('Age', 'Purchase Amount (USD)'), we'll impute with the median.
# For categorical columns ('Gender', 'Item Purchased', 'Category', 'City', 'Season', 'Subscription Status', 'Payment Method', 'Shipping Type', 'Discount Applied', 'Promo Code Used', 'Previous Purchases', 'Preferred Payment Method'), we'll use the mode.

# Age: Impute with median
df['Age'].fillna(df['Age'].median(), inplace=True)

# Income (assuming 'Purchase Amount (USD)' is the closest to 'income' for this dataset, or 'Age' as no income is present): No 'income' column, using 'Purchase Amount (USD)'.
# If 'income' was explicitly present, the approach would be similar to 'Age'.
df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].median(), inplace=True)

# City: Impute with mode
df['Location'].fillna(df['Location'].mode()[0], inplace=True)

print("\nDataFrame Info after handling missing values:")
df.info()

print("\nNumber of missing values after imputation:")
print(df.isnull().sum())

```

DataFrame Info after handling missing values:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID            3900 non-null   int64
1   Age                    3900 non-null   int64
2   Gender                 3900 non-null   object

```

Terminal

1:47 PM Python 3

```
10 Review Rating      3900 non-null    float64
11 Subscription Status 3900 non-null    object
12 Payment Method      3900 non-null    object
13 Shipping Type       3900 non-null    object
14 Discount Applied    3900 non-null    object
15 Promo Code Used     3900 non-null    object
16 Previous Purchases  3900 non-null    int64
17 Preferred Payment Method 3900 non-null object
18 Frequency of Purchases 3900 non-null object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

Number of missing values after imputation:

```
Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD) 0
Location         0
Size            0
Color           0
Season          0
Review Rating    0
Subscription Status 0
Payment Method   0
Shipping Type    0
Discount Applied 0
Promo Code Used  0
Previous Purchases 0
Preferred Payment Method 0
Frequency of Purchases 0
dtype: int64
```

/tmp/ipykernel_9687/97586187.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

Terminal

1:47 PM Python 3

Next, we will remove any duplicate records from the dataset.

```
[6]
# Remove duplicate records
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
final_rows = df.shape[0]

print(f"Number of rows before removing duplicates: {initial_rows}")
print(f"Number of rows after removing duplicates: {final_rows}")
print(f"Number of duplicate records removed: {initial_rows - final_rows}")
```

```
Number of rows before removing duplicates: 3900
Number of rows after removing duplicates: 3900
Number of duplicate records removed: 0
```

Now, let's standardize text values, specifically by converting city names to lowercase.

```
[7]
# Standardize text values (e.g., city names in lowercase)
df['Location'] = df['Location'].str.lower()

print("\nUnique city names after standardization:")
print(df['Location'].unique()[:10]) # Display first 10 unique cities
```

```
Unique city names after standardization:
['kentucky' 'maine' 'massachusetts' 'rhode island' 'oregon' 'wyoming'
 'montana' 'louisiana' 'west virginia' 'missouri']
```

Finally, we will encode categorical variables ('Gender' and 'Location'). We'll use `LabelEncoder` for 'Gender' and 'Location' to convert them into numerical representations, which is suitable for many machine learning models.

```
from sklearn.preprocessing import LabelEncoder
```

```
# Encode categorical variables
le = LabelEncoder()
```

```
# Encode 'Gender'
df['Gender_Encoded'] = le.fit_transform(df['Gender'])
print(f"Original Gender categories: {le.classes_}")
```

```
# Encode 'Location'
df['Location_Encoded'] = le.fit_transform(df['Location'])
print(f"Original Location categories (first 10): {le.classes_[:10]}...")
```

```
print("\nDataFrame Head after encoding categorical variables:")
display(df.head())
```

```
Original Gender categories: ['Female' 'Male']
Original location categories (first 10): ['alabama' 'alaska' 'arizona' 'arkansas' 'california' 'colorado'
 'connecticut' 'delaware' 'florida' 'georgia']...
```

DataFrame Head after encoding categorical variables:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	...	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	kentucky	L	Gray	Winter	...	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortr
1	2	19	Male	Sweater	Clothing	64	maine	L	Maroon	Winter	...	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortr
2	3	50	Male	Jeans	Clothing	73	massachusetts	S	Maroon	Spring	...	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	V

Terminal

1:47 PM Python 3

```

print("Final Cleaned DataFrame Info:")
df.info()

print("\nFinal Cleaned DataFrame Head:")
display(df.head())

```

```

... Final Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   object
3   Item Purchased                      3900 non-null   object
4   Category                            3900 non-null   object
5   Purchase Amount (USD)               3900 non-null   int64
6   Location                            3900 non-null   object
7   Size                                3900 non-null   object
8   Color                               3900 non-null   object
9   Season                              3900 non-null   object
10  Review Rating                       3900 non-null   float64
11  Subscription Status                 3900 non-null   object
12  Payment Method                     3900 non-null   object
13  Shipping Type                      3900 non-null   object
14  Discount Applied                   3900 non-null   object
15  Promo Code Used                    3900 non-null   object
16  Previous Purchases                 3900 non-null   int64
17  Preferred Payment Method           3900 non-null   object
18  Frequency of Purchases              3900 non-null   object
19  Gender_Encoded                     3900 non-null   int64
20  Location_Encoded                   3900 non-null   int64

```

```

12 Payment Method          3900 non-null   object
13 Shipping Type           3900 non-null   object
14 Discount Applied        3900 non-null   object
15 Promo Code Used         3900 non-null   object
16 Previous Purchases      3900 non-null   int64
17 Preferred Payment Method 3900 non-null   object
18 Frequency of Purchases  3900 non-null   object
19 Gender_Encoded          3900 non-null   int64
20 Location_Encoded        3900 non-null   int64
dtypes: float64(1), int64(6), object(14)
memory usage: 640.0+ KB

```

Final Cleaned DataFrame Head:

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	...	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases	
0	1	55	Male	Blouse	Clothing	53	kentucky	L	Gray	Winter	...	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	maine	L	Maroon	Winter	...	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	massachusetts	S	Maroon	Spring	...	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	rhode island	M	Maroon	Spring	...	Yes	PayPal	Next Day Air	Yes	Promo	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	oregon	M	Turquoise	Spring	...	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

5 rows x 21 columns

Task 2 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```
import pandas as pd  
data = pd.read_csv("sales_data.csv")  
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Load the sales dataset. Assuming it's uploaded to the Colab environment.
# If using a remote URL, replace '/content/sales_data.csv' with the direct URL.
data_url = "/content/sales_data.csv"
sales_df = pd.read_csv(data_url)

print("Original Sales DataFrame Info:")
sales_df.info()

print("\nOriginal Sales DataFrame Head:")
display(sales_df.head())

```

```

... Original Sales DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Product_ID                           1000 non-null   int64
1   Sale_Date                            1000 non-null   object
2   Sales_Rep                            1000 non-null   object
3   Region                               1000 non-null   object
4   Sales_Amount                         1000 non-null   float64
5   Quantity_Sold                       1000 non-null   int64
6   Product_Category                    1000 non-null   object
7   Unit_Cost                           1000 non-null   float64
8   Unit_Price                          1000 non-null   float64
9   Customer_Type                       1000 non-null   object
10  Discount                            1000 non-null   float64
11  Payment_Method                      1000 non-null   object

```

Terminal

```

# Convert 'Date' column to datetime format
sales_df['Sale_Date'] = pd.to_datetime(sales_df['Sale_Date'])

# Create new features: month, year, day of week
sales_df['Month'] = sales_df['Sale_Date'].dt.month
sales_df['Year'] = sales_df['Sale_Date'].dt.year
sales_df['DayOfWeek'] = sales_df['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6

print("\nDataFrame Head with new date features:")
display(sales_df.head())

```

... DataFrame Head with new date features:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Regio
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	

Deletion use Ctrl+M Z or the Undo option in the Edit menu


```
# Normalize 'Sales_Amount' column
scaler = MinMaxScaler()
sales_df['Sales_Amount_Normalized'] = scaler.fit_transform(sales_df[['Sales_Amount']])

print("\nDataFrame Head with normalized sales amount:")
display(sales_df.head())
```

... DataFrame Head with normalized sales amount:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	
2	1015	2023-04-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	

Finally, we will detect and handle outliers in the 'Sales_Amount' column using the Interquartile Range (IQR) method. Outliers will be capped

```
# Detect and handle outliers in 'Sales_Amount' using IQR
Q1 = sales_df['Sales_Amount'].quantile(0.25)
Q3 = sales_df['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

sales_df['Sales_Amount_Capped'] = sales_df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

print(f"\nOutlier Detection (IQR Method) for Sales_Amount:")
print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Lower Bound: {lower_bound}, Upper Bound: {upper_bound}")
print(f"Number of outliers below lower bound: {(sales_df['Sales_Amount'] < lower_bound).sum()}")
print(f"Number of outliers above upper bound: {(sales_df['Sales_Amount'] > upper_bound).sum()}")

print("\nDataFrame Head with capped sales amount:")
display(sales_df.head())
```

... Outlier Detection (IQR Method) for Sales_Amount:
Q1: 2550.2975, Q3: 7507.445000000001, IQR: 4957.147500000001
Lower Bound: -4885.423750000002, Upper Bound: 14943.166250000002
Number of outliers below lower bound: 0
Number of outliers above upper bound: 0

DataFrame Head with capped sales amount:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	

Terminal 1:47 PM Python 3

```
▶ print("Final Preprocessed Sales DataFrame Info:")
sales_df.info()

print("\nFinal Preprocessed Sales DataFrame Head:")
display(sales_df.head())
```

```
*** Final Preprocessed Sales DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Product_ID                           1000 non-null   int64
1   Sale_Date                             1000 non-null   datetime64[ns]
2   Sales_Rep                             1000 non-null   object
3   Region                                1000 non-null   object
4   Sales_Amount                          1000 non-null   float64
5   Quantity_Sold                         1000 non-null   int64
6   Product_Category                      1000 non-null   object
7   Unit_Cost                             1000 non-null   float64
8   Unit_Price                            1000 non-null   float64
9   Customer_Type                         1000 non-null   object
10  Discount                              1000 non-null   float64
11  Payment_Method                        1000 non-null   object
12  Sales_Channel                         1000 non-null   object
13  Region_and_Sales_Rep                  1000 non-null   object
14  Month                                 1000 non-null   int32
15  Year                                  1000 non-null   int32
16  DayOfWeek                             1000 non-null   int32
17  Sales_Amount_Normalized                1000 non-null   float64
18  Sales_Amount_Capped                    1000 non-null   float64
dtypes: datetime64[ns](1), float64(6), int32(3), int64(2), object(7)
memory usage: 136.8+ KB
```

Task 3 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

<https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset>

Expected Output:

- A preprocessed dataset ready for machine learning models.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Load the healthcare dataset from the provided link
data_url = "/content/dirty_v3_path.csv"
healthcare_df = pd.read_csv(data_url)

print("Original Healthcare DataFrame Info:")
healthcare_df.info()

print("\nOriginal Healthcare DataFrame Head:")
display(healthcare_df.head())
```

```
... Original Healthcare DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Age                   25500 non-null  float64
 1   Gender                25500 non-null  object
 2   Medical Condition     25500 non-null  object
 3   Glucose               25500 non-null  float64
 4   Blood Pressure        25500 non-null  float64
 5   BMI                   30000 non-null  float64
 6   Oxygen Saturation     30000 non-null  float64
 7   LengthOfStay          30000 non-null  int64
 8   Cholesterol           30000 non-null  float64
 9   Triglycerides         30000 non-null  float64
10   HbA1c                 30000 non-null  float64
11   Smoking               30000 non-null  int64
12   Alcohol               30000 non-null  int64
13   Physical Activity     30000 non-null  float64
```

```
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB
```

Original Healthcare DataFrame Head:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	Physical Activity	Diet Score	Family History	Stress Level	Sleep Hours	random_notes
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	0	-0.20	3.54	0	5.07	6.05	
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	0	8.12	5.90	0	5.87	7.72	
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2	214.94	165.35	5.60	0	0	5.01	4.65	1	3.09	4.82	
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	6.92	0	0	3.16	3.37	0	3.01	5.33	
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	1	3.56	3.40	0	6.38	6.64	

Task 3: Healthcare Data Preparation

Let's start by handling missing values in the `blood_pressure` and `cholesterol` columns. For numerical features, median imputation is a robust strategy.

```
[33] ✓ Os
# Handle missing values in 'blood_pressure' and 'cholesterol'
# Using median imputation for numerical features
healthcare_df['blood_pressure'] = healthcare_df['blood_pressure'].fillna(healthcare_df['blood_pressure'].median())
healthcare_df['cholesterol'] = healthcare_df['cholesterol'].fillna(healthcare_df['cholesterol'].median())

print("\nHealthcare DataFrame Info after handling missing values:")
healthcare_df.info()

print("\nNumber of missing values after imputation:")
```

Healthcare DataFrame Info after handling missing values:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 40 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    25500 non-null  float64
1   Gender                                25500 non-null  object
2   Medical Condition                     25500 non-null  object
3   Glucose                                25500 non-null  float64
4   Blood Pressure                        30000 non-null  float64
5   BMI                                    30000 non-null  float64
6   Oxygen Saturation                     30000 non-null  float64
7   LengthOfStay                          30000 non-null  int64
8   Cholesterol                           30000 non-null  float64
9   Triglycerides                         30000 non-null  float64
10  HbA1c                                 30000 non-null  float64
11  Smoking                               30000 non-null  int64
12  Alcohol                               30000 non-null  int64
13  Physical Activity                     30000 non-null  float64
14  Diet Score                            30000 non-null  float64
15  Family History                        30000 non-null  int64
16  Stress Level                          30000 non-null  float64
17  Sleep Hours                           30000 non-null  float64
18  random_notes                          30000 non-null  object
19  noise_col                             30000 non-null  float64
20  Gender_Encoded                        30000 non-null  int64
21  Medical Condition_Encoded             30000 non-null  int64
22  random_notes_Encoded                  30000 non-null  int64
23  Age_Scaled                            25500 non-null  float64
24  Glucose_Scaled                        25500 non-null  float64
25  Blood Pressure_Scaled                 30000 non-null  float64
26  BMI_Scaled                            30000 non-null  float64
27  Oxygen Saturation_Scaled              30000 non-null  float64
```

Terminal

```

Alcohol_Scaled          0
Physical_Activity_Scaled 0
Diet_Score              0
...
Family_History          0
Stress_Level            0
Sleep_Hours             0
random_notes            0
noise_col               0
Gender_Encoded          0
Medical_Condition_Encoded 0
random_notes_Encoded    0
Age_Scaled              4500
Glucose_Scaled           4500
Blood_Pressure_Scaled    0
BMI_Scaled              0
Oxygen_Saturation_Scaled 0
LengthOfStay_Scaled     0
Cholesterol_Scaled       0
Triglycerides_Scaled     0
HbA1c_Scaled            0
Smoking_Scaled          0
Alcohol_Scaled          0
Physical_Activity_Scaled 0
Diet_Score_Scaled        0
Family_History_Scaled    0
Stress_Level_Scaled      0
Sleep_Hours_Scaled       0
noise_col_Scaled        0
dtype: int64

```

Next, we will encode categorical features. We'll identify object-type columns and apply `LabelEncoder` to convert them into numerical representations.

Encode categorical features

```

# Import necessary libraries

```

Encode categorical features

```

le = LabelEncoder()
categorical_cols = healthcare_df.select_dtypes(include='object').columns

for col in categorical_cols:
    healthcare_df[col + '_Encoded'] = le.fit_transform(healthcare_df[col])
    print(f"Original categories for '{col}': {le.classes_}")

print("\nDataFrame Head after encoding categorical variables:")
display(healthcare_df.head())

```

```

...
Original categories for 'Gender': ['Female' 'Male' nan]
Original categories for 'Medical Condition': ['Arthritis' 'Asthma' 'Cancer' 'Diabetes' 'Healthy' 'Hypertension'
'Obesity' nan]
Original categories for 'random_notes': ['###' '??' 'ipsum' 'lorem']

```

DataFrame Head after encoding categorical variables:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	...	Physical Activity	Diet Score	Family History	Stress Level	Sleep Hours	random_notes	noise_col
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	...	-0.20	3.54	0	5.07	6.05	lorem	-137.0572
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	...	8.12	5.90	0	5.87	7.72	ipsum	-11.2306
2	50.0	NaN	Asthma	95.24	138.32	22.53	90.31	2	214.94	165.35	...	5.01	4.65	1	3.09	4.82	ipsum	98.3311
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	...	3.16	3.37	0	3.01	5.33	lorem	44.1871
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	...	3.56	3.40	0	6.38	6.64	lorem	44.8314

5 rows x 23 columns

Now, we will scale numerical features using `MinMaxScaler` to bring them into a standard range (0 to 1). We'll exclude the original categorical columns and the newly encoded ones, as well as `id`.

```
# Scale numerical features using MinMaxScaler
scaler = MinMaxScaler()

# Identify numerical columns to scale (excluding original categorical and 'id')
# Also exclude newly created encoded columns from scaling
all_cols = set(healthcare_df.columns)
encoded_cols = {col for col in healthcare_df.columns if '_Encoded' in col}
original_categorical_cols = set(categorical_cols)

# Numerical columns are those that are not object type, not encoded, and not 'id'
numerical_cols = {col for col in healthcare_df.select_dtypes(include=np.number).columns
                  if col not in encoded_cols and col != 'id'}

for col in numerical_cols:
    healthcare_df[col + '_Scaled'] = scaler.fit_transform(healthcare_df[[col]])

print("\nDataFrame Head after scaling numerical features:")
display(healthcare_df.head())
```

... DataFrame Head after scaling numerical features:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	...	Triglycerides_Scaled	HbA1c_Scaled	Smoking_Scaled	Alcohol_Scaled
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	...	0.524877	0.476872	0.0	0.0
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	...	0.342102	0.179515	0.0	0.0
2	50.0	NaN	Asthma	95.24	138.32	22.53	90.31	2	214.94	165.35	...	0.423050	0.255507	0.0	0.0

Terminal 1:47 PM Python 3

Finally, we will split the dataset into training and testing sets. We'll use the `target` column (assuming 'heart_disease' is the target variable) as our dependent variable (`y`), and all other relevant preprocessed features as our independent variables (`X`).

```
# Split dataset into training and testing sets

# Define the target variable (assuming 'LengthOfStay' is the intended target based on data content)
target_column = 'LengthOfStay'
y = healthcare_df[target_column]

# Identify feature columns for X
# Include all _Encoded categorical columns and _Scaled numerical columns,
# but exclude the scaled version of the target variable itself.
X_cols = [col for col in healthcare_df.columns if col.endswith('_Encoded') or col.endswith('_Scaled')]
if f'{target_column}_Scaled' in X_cols:
    X_cols.remove(f'{target_column}_Scaled')

X = healthcare_df[X_cols]

# Split the data, removing 'stratify=y' as LengthOfStay is likely a continuous target for regression.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())
```

... Shape of X_train: (24000, 19)

Task 4 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

<https://www.kaggle.com/code/vishantmathur/employee-salary>

Expected Output:

- A structured dataset ready for HR analytics.

The image displays three sequential screenshots of a Jupyter Notebook interface, illustrating the steps to clean and preprocess an employee salary dataset.

Step 1: Loading and Initial Cleaning

The first screenshot shows the initial data loading and cleaning steps. The file explorer on the left lists the dataset files: `sample_data`, `Salary_Data.csv`, `processed_employee_salary.csv`, `step1_missing_values_handled.csv`, `step2_outliers_removed.csv`, and `step3_department_cleaned.csv`. The code cell [12] imports `pandas` and `numpy`, reads the data from `Salary_Data.csv`, and fills missing values in numeric columns with the median and in categorical columns with the mode. It then saves the intermediate file `step1_missing_values_handled.csv`.

```
[12] import pandas as pd
import numpy as np

df = pd.read_csv("/content/Salary_Data.csv")

# Fill numeric columns with median
num_cols = df.select_dtypes(include=np.number).columns
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Fill categorical columns with mode
cat_cols = df.select_dtypes(include='object').columns
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("Missing values handled successfully")

# Save intermediate file
df.to_csv("step1_missing_values_handled.csv", index=False)
```

Step 2: Outlier Removal

The second screenshot shows the removal of outliers. The code cell [13] reads the intermediate file `step1_missing_values_handled.csv`, calculates the IQR, and removes outliers based on the 1.5 * IQR rule. It then saves the intermediate file `step2_outliers_removed.csv`.

```
[13] import pandas as pd

df = pd.read_csv("step1_missing_values_handled.csv")

salary_col = "Salary" # change if needed

Q1 = df[salary_col].quantile(0.25)
Q3 = df[salary_col].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df = df[(df[salary_col] >= lower) & (df[salary_col] <= upper)]

print("Outliers removed successfully")

# Save intermediate file
df.to_csv("step2_outliers_removed.csv", index=False)
```

Step 3: Department Standardization and Label Encoding

The third screenshot shows the standardization of department names and the application of label encoding. The code cell [15] reads the intermediate file `step2_outliers_removed.csv`, standardizes department names, and saves the intermediate file `step3_department_cleaned.csv`. The code cell [18] imports `LabelEncoder` from `sklearn.preprocessing`, fits the encoder on the `Department` column, and applies the transformation. It then saves the final dataset `processed_employee_salary.csv`.

```
[15] import pandas as pd

df = pd.read_csv("/content/step2_outliers_removed.csv")

# df['Department'] = df['Department'].str.lower().str.strip() # Removed as 'Department' column does not exist
print("Department names standardization skipped as column not found.")

# Save intermediate file
df.to_csv("step3_department_cleaned.csv", index=False)

[18] import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("/content/step3_department_cleaned.csv")

le = LabelEncoder()

# The following lines are commented out as 'Department' and 'Location' columns do not exist in the DataFrame.
# df['Department_Encoded'] = le.fit_transform(df['Department'])
# df['Location_Encoded'] = le.fit_transform(df['Location'])

print("Label encoding for non-existent columns skipped.")

# Save final dataset
df.to_csv("processed_employee_salary.csv", index=False)
```